Table of Contents

Table Overview of Workflows – UPDATED -04/21/2026 ..... *2*
01_preprocessing.smk **2**
02_variantcalling_manual.smk **3**
02_variantcalling_BCF_VEP.smk **4**
Workflows Formatted like Galaxy *5*
01_preprocessing.smk **5**
02_variantcalling_manual.smk **5**
02_variantcalling_BCF_VEP.smk **6**
Main Changes to 02_variantcalling_manual.smk *7*
Addressing Red Flags for the Rules **7**
Improvements to Workflow **12**
Implementing Local Scratch into Workflow 12
Testing Ceph with Workflow 14
Comparing gscratch with Normal Performance (while waiting for local scratch to work) 15
Implementing CleanUp 15
Implementing Metrics File 16
Implementing New Method for Annovar 17
Addressing Issue with COSMIC 18
Implementing New Script for Adding Annotations 18
Running the VC Workflows *20*
Pulling Data from Ceph **20**
Config File Breakdown **20**
config_variantcalling.json 20
config_vc_vep.json 21
Confirm Chromosome Format 22
Cluster Generic Profile Breakdown **22**
config/cluster/config.v8+.yaml 22
config/cluster_updated/config.v8+.yaml 23
Conda Environments **23**
HOP 23
vep_HOP 24
Launching the Workflow **24**
Case Studies **25**
Other Information **26**

Table Overview of Workflows – UPDATED -04/21/2026

01_preprocessing.smk

Step + Purpose	Input	Output	Occurs	Elapsed	CPUEff
fastqc Generate a FASTQC report per paired reads.	*_1.fq.gz, *_2.fq.gz	FASTQC/*_1_fastqc.html, FASTQC/*_2_fastqc.html, FASTQC/*_1_fastqc.zip, FASTQC/*_2_fastqc.zip	Per sample	Avg: 00:34:12 Max: 02:59:58 Min: 00:05:04	Avg: 77.28% Max: 95.6% Min: 36.14%
trim Trim Illumina adapters off of paired reads.	*_1.fq.gz, *_2.fq.gz	Trimmed/*_trimmed_1.fq.gz, Trimmed/*_trimmed_2.fq.gz	Per sample	Avg: 00:24:16 Max: 03:23:15 Min: 00:04:29	Avg: 44.4% Max: 81.4% Min: 12.7%
align Align reads to hg19 genome.	Trimmed/*_trimmed_1.fq.gz, Trimmed/*_trimmed_2.fq.gz	Aligned/*.aligned.bam	Per sample	Avg: 01:38:33 Max: 03:32:09 Min: 00:40:05	Avg: 77.8% Max: 87.3% Min: 48.43%

check_rg Add read group information if it doesn't exist.	Aligned/*.aligned.bam	Aligned/*.rg_exists.txt	Per sample	Avg: 00:46:22 Max: 01:38:33 Min: 00:06:43	Avg: 89.5% Max: 95.8% Min: 78.47%
deduplicate Remove duplicates and create a BAM index	Aligned/*.aligned.bam, Aligned/*.rg_exists.txt	Deduped/*.sort.bam, Deduped/*.sort.bam.bai, Deduped/*.dedup_metrics.txt	Per sample	Avg: 00:57:23 Max: 03:36:01 Min: 00:06:27	Avg: 88.03% Max: 97.39% Min: 47.74%

02_variantcalling_manual.smk

Step + Purpose	Input	Output	Occurs	Elapsed	CPUEff
readcount Count reads from bam file and add library name	*.sort.bam *.sort.bam.bai	ReadCounts_Vafs/*.readcount, Metrics/*.metrics	Per sample	-	-
calc_freq Adds column w/ calculated VAF and minimal QC of min depth of 20X and min vaf of 0.01%	ReadCounts_Vafs/*.readcount, Metrics/*.metrics	ReadCounts_Vafs/*.all_freq, Metrics/*.metrics	Per sample	Avg: 00:00:57 Max: 00:05:05 Min: 00:00:11	Avg: 73.3% Max: 99.7% Min: 8.53%
filt Filters for min depth of 50X and min vaf of 0.05%	ReadCounts_Vafs/*.all_freq, Metrics/*.metrics	ReadCounts_Vafs/ *.#min_d_#percentmin_vaf.freq, Metrics/*.metrics	Per sample	Avg: 00:00:16 Max: 00:02:30 Min: 00:00:01	Avg: 28.6% Max: 111% Min: 1.09%
format_annov Reorganized into format annovar accepts	ReadCounts_Vafs/ *.#min_d_#percentmin_vaf.freq	Annovar/*.freq.annovar_file	Per sample	Avg: 00:12:45 Max: 02:17:09 Min: 00:00:14	Avg: 95.4% Max: 99.5% Min: 54%
do_annovar Perform annovar to obtain functional annotations	Annovar/*.freq.annovar_file	Annovar/*.hg19_multianno.txt	Per sample	Avg: 00:01:13 Max: 00:05:29	Avg: 81% Max: 98.6%

				Min: 00:00:11	Min: 9.01%
add_annotations Adds columns from annovar: class of variant, gene, function	ReadCounts_Vafs/ *.#min_d_#percentmin_vaf.freq, Annovar/*.hg19_multianno.txt, Metrics/*.metrics	Annotated/*.annotated_readcounts, Metrics/*.metrics	Per sample	Avg: 00:54:37 Max: 06:28:58 Min: 00:00:31	Avg: 98.1% Max: 99.6% Min: 57.2%
merge Combine all variant data per library into one df	Annotated/ *.annotated_readcounts	merged_final.csv	Once	00:02:56	25.11%
final_filt Filter out synonymous, VAF must be between 0.2 – 45%, min supporting reads is 5, and min BQ is 30	merged_final.csv	{rundate}_merged_filt_nonsyn_0.2 to45percent _ 3supp_BQ30	Once	00:02:08	59.48%

*EVF = exonic_variant_function
*Highlighted in Yellow = Default Steps Kept during Cleanup
*Boxes Highlighted in Orange = Metrics No Longer Apply due to major change on 08/07

02_variantcalling_BCF_VEP.smk

Step + Purpose	Input	Output	Occurs	Elapsed	CPUEff
bcf_call Call, normalize, and index SNV/indel variants.	*.sort.bam *.sort.bam.bai	bcf_calls/*.raw.vcf.gz, bcf_calls/*.raw.vcf.gz.csi	Per sample	Avg: 0:03:11 Max: 0:08:13 Min: 0:01:08	Avg: 44.9% Max: 87.89% Min: 17.58%
bcf_filt Filter variants by min variant quality, read depth and genotype quality.	bcf_calls/*.raw.vcf.gz	bcf_calls/*_VarQ##X_GQ#.vcf.gz, bcf_calls/*_VarQ##X_GQ#.vcf.gz.csi	Per sample	Avg: 0:00:17 Max: 0:00:41 Min: 0:00:03	Avg: 14.42% Max: 54.6% Min: 2.9%
prefilt_w_gnomad Annotate with gnomad and filter by max AF.	bcf_calls/*_VarQ##X_GQ#.vcf.gz	bcf_calls/*_MaxAF# _VarQ# #X_GQ#.vcf.gz, bcf_calls/*_MaxAF# _VarQ# #X_GQ#.vcf.gz.csi	Per sample	Avg: 0:09:59 Max: 0:14:45 Min: 0:03:07	Avg: 94.38% Max: 98.43% Min: 64.82%
annotate_vep Uses VEP to add gene, transcript, protein and	bcf_calls/*_MaxAF# _VarQ# #X_GQ#.vcf.gz	annotated_vep/*.csq.vcf.gz, annotated_vep/*.csq.vcf.gz_ summary.html, annotated_vep/*.csq.vcf.gz.csi	Per sample	Avg: 0:01:03 Max: 0:03:03	Avg: 3.43% Max: 14.57%

functional annotations.				Min: 0:00:32	Min: 0.93%

*Boxes Highlighted in Orange = Metrics No Longer Apply due to major change on 04/28

Workflows Formatted like Galaxy

*All user-specified parameters are modified in the config files. These workflows are in Snakemake.

01_preprocessing.smk

02_variantcalling_manual.smk

1. readcount (bam-readcount)

Inputs: Sorted Bam (*.sort.bam), Sorted Bam Index (*.sort.bam.bai)
Non-default parameters: config[max_depth], config[regions]
Usage: Calculates read counts for bam file with minor filtering of user-specified max depth. Uses user-specified regions to target which regions to report readcount information.

2. calc_freq (awk)

Inputs: readcount (1)
Non-default parameters: NA
Usage: Calculates VAF and applies very minimal default filtering of depth (>= 20), vaf (>= 0.01%) and supporting reads (>= 3).

3. filt (awk)

Inputs: calc_freq (2)
Non-default parameters: config[min_depth, config[min_vaf]
Usage: Performs filtering using user-specified minimum depth and minimum vaf.

4. format_annov (pandas operations)

Inputs: filt (3)
Non-default parameters: NA
Usage: Reorganize into format annovar tool accepts.

5. do_annovar (annovar)

Inputs: format_annov (4)
Non-default parameters: config[addl_db], config[build]
Usage: Perform annovar to obtain functional annotations. Users can specify additional databases for annotation and build version.

6. add_annotations (pandas operations)

Inputs: do_annovar (5), filt (3)
Non-default parameters: NA
Usage: Add variant class, gene, function and other information from annovar's annotations.

7. merge (bash operations)

Inputs: add_annotations (6)
Non-default parameters: NA
Usage: Merge annotated variant information across libraries into one file.

8. final_filt (awk)

Inputs: merge (7)
Non-default parameters: config[max_vaf], config[min_vaf], config[minBQ], config[min_supp_reads]
Usage: Perform additional filtering given user-specified minimum vaf, maximum vaf, min base quality, minimum supporting reads. Also, keep only nonsynonymous variants.

02_variantcalling_BCF_VEP.smk

1. bcf_call (bcftools mpileup, call, norm, index)

Inputs: Sorted Bam (*.sort.bam), Sorted Bam Index (*.sort.bam.bai)
Non-default parameters: config[regions], config[ref], config[min_map_q], config[min_base_q]
Usage: Calls SNV and indel variants using bcftools mpileup with user-defined reference, quality filters and target regions. Outputs a normalized, compressed, and indexed VCF.

2. bcf_filt (bcftools filter)

Inputs: bcf_call (1)
Non-default parameters: config[min_var_q], config[min_depth], config[min_gq]
Usage: Applies minimal filtering to exclude calls below user-defined thresholds for variant quality, depth, and genotype quality.

3. prefilt_w_gnomad (bcftools annotate, filter, index)

Inputs: bcf_filt (2)
Non-default parameters: config[max_af]
Usage: Annotates calls with population allele frequency from gnomAD and filters out those exceeding a user-defined AF threshold.

4. annotate_vep (vep)

Inputs: prefilt_w_gnomad (3)
Non-default parameters: config[vep_dir], config[build], config[t]
Usage: Annotates calls using VEP, using a preloaded VEP cache directory and a reference FASTA file.

Main Changes to 02_variantcalling_manual.smk

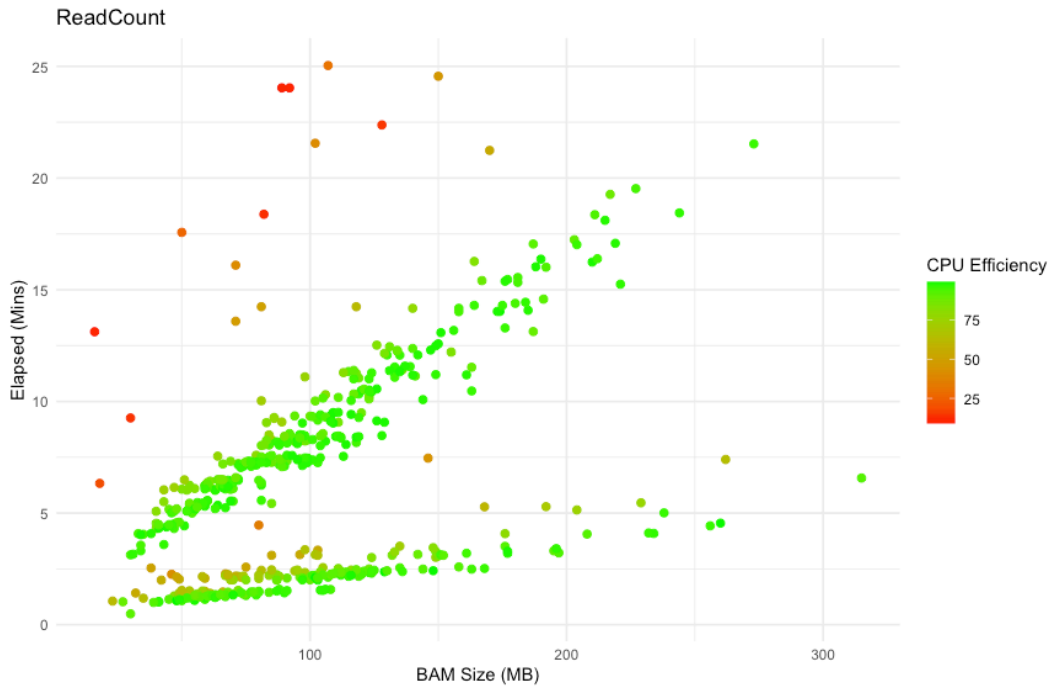
These changes are reflected in the above table:

1. Implement Local Scratch for entire workflow.
2. Merge smaller rules (ex. Rule: Readcount + incl_FN)
3. Is there a difference between min cov and min depth?
    a. readcount tool calculates depth as the total number of bases present at each position
    b. calc_freq filter the depth by the min coverage variable (50 in this case)
    c. initial_filt filters by depth again this time using the min depth variable (50 in this case)
    d. ^ looks like both min cov variable and min depth variable are the same???
    e. They should be the same, one can be removed
4. Implement clean up function for workflow, with some default steps whose output will be kept. Must include way in config file to specify steps to keep and whether clean up should occur.
5. Implement metrics file for workflow. This serves to keep track of where the sample is in the workflow and to monitor how the number of variants change as the sample is QCed, filtered, annotated etc. These metrics files can be added to make an enhanced metadata file.
6. The way we run annovar and add annotations has been **CHANGED**. We are now using table_annovar.pl instead of just annotate_variation.pl. A new script has been written to add the annotations to the data to create *.annotated_readcounts. This will allow us the flexibility to add additional databases we want annotations for. As a result, prior stats in the do_annovar and add_annotations steps are no longer relevant as indicated in the table.

Want to know why these changes? Refer to below sections.

Addressing Red Flags for the Rules

readcount: Significant range of elapsed (49 secs – 25 mins) and of CPU eff (8% - 99%)

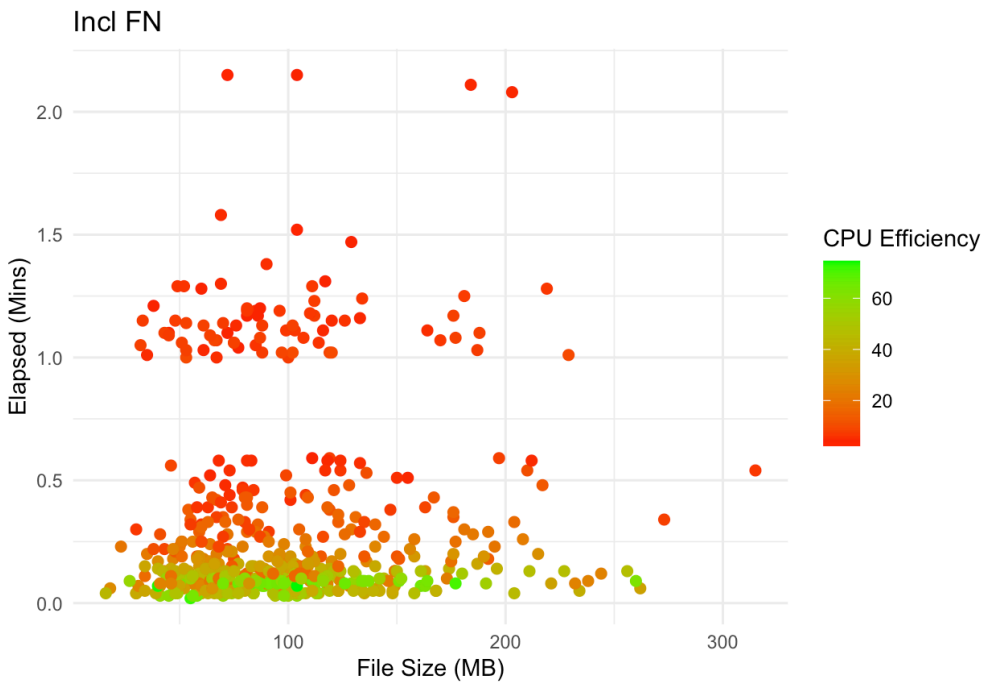


There are 2 linear trends whereas when the BAM size increases, elapsed increases as well. Those following the trends have moderate to high CPU efficiency. Those that don't follow these trends have low CPU efficiency < 25%. Strangely, these outliers are also not very big, all are < 150 MB.

Taking a look further, I isolated a data point with size 93 MB and 98% CPU Efficiency and compared it to one of the outliers of size 92 MB and 10% CPU Efficiency. The outlier took a lot longer: 24 minutes vs 7 minutes. I attempted to find the reason behind this difference: There is a significant difference between TotalCPU and CPUTime for the outlier which is an indication of a lot of wasted time where the CPU was doing nothing and waiting for I/O. **Perhaps this can be corrected by running on local scratch?**

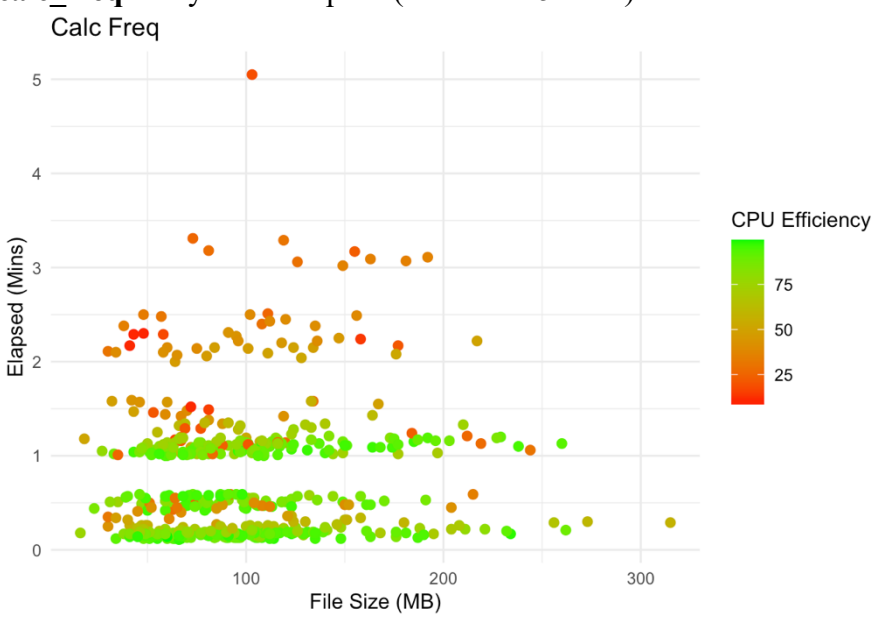
Size	TotalCPUSec	CPUTimeSec	CPUEff
93	469.47	478	98.22
92	146.33	1444	10.13

incl_FN: Very small elapsed (2 secs to 2 mins)

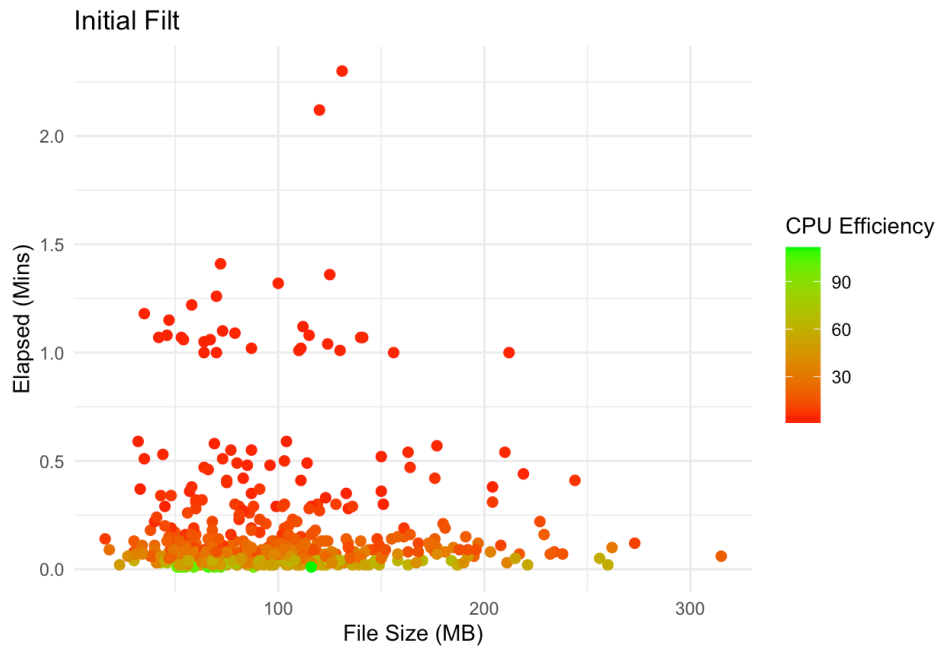


This is a really small job step so **is it necessary to make it its own job and dedicate 1 whole CPU to it?** The CPU efficiency is overall very low because most of the CPU is not being used as it's such a small job. **What if we combined this with the calc_freq step?**

calc_freq: Very small elapsed (11 secs to 5 mins)

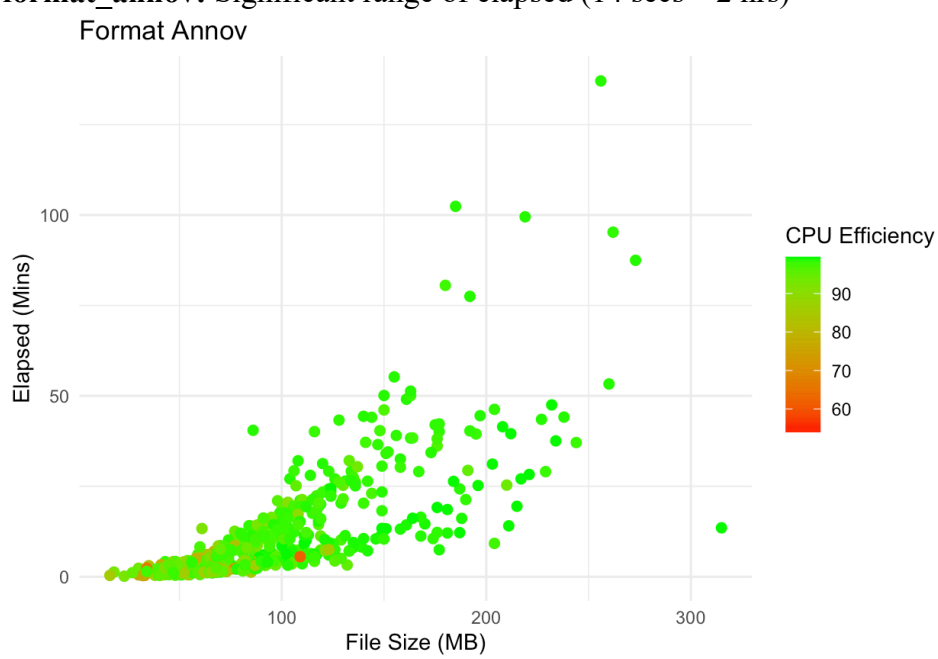


ıl, those with low CPU
all, quick job so **is it**
Combining it with
e 1 CPU.



Similar to both incl_FN and calc_freq, this is the smallest job with the worst CPU efficiency because it has minimal computation. It would be **ideal to combine it with calc_freq and/or incl_FN so that one CPU is shared.** Unless, there is a reason why we would want to keep this separate filtered file.

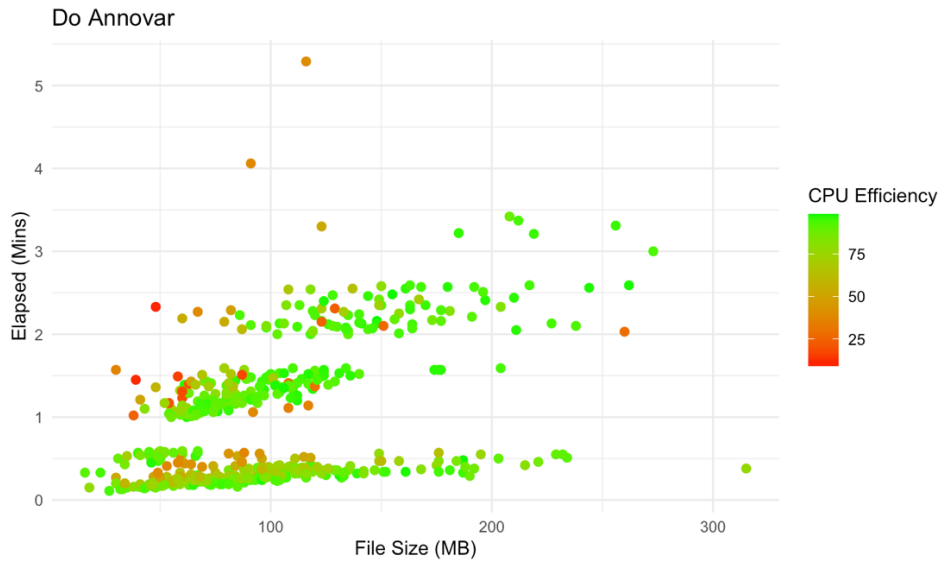
format_annov: Significant range of elapsed (14 secs – 2 hrs)



Most points follow the general trend: as the size of the file increases, so does the elapsed time. There are also high CPU efficiencies in general, all > 50%. **There aren't that many large files**

that are taking > 1 hour, so may not be worth upping CPUs here even though there are mainly high CPU efficiencies present **as it might not make that significant of a difference.**

do_annovar: Significant range of CPU eff (9-98%)



This graph is very similar to what we observed in readcount, where there are a few strange outliers from the general trends. These outliers are also the ones with low CPU efficiency. Similarly to readcount, I investigated by comparing 2 data points : one with 46 MB and a CPU efficiency of 97.56% and the other with 48 MB and CPU efficiency of 9%. There is a significant time difference between CPUTime and TotalCPU for the job wih low CPU efficiency.

Size	TotalCPUSec	CPUTimeSec	CPUEff
46	43.90	45	97.56
48	13.79	153	9.01

This is an indication of a lot of wasted time where the CPU was doing nothing and waiting for I/O. **Perhaps this can be corrected by running on local scratch?**

Add_annotations: Significant range of elapsed (31 secs - 6 ½ hours)

ould
CPUs

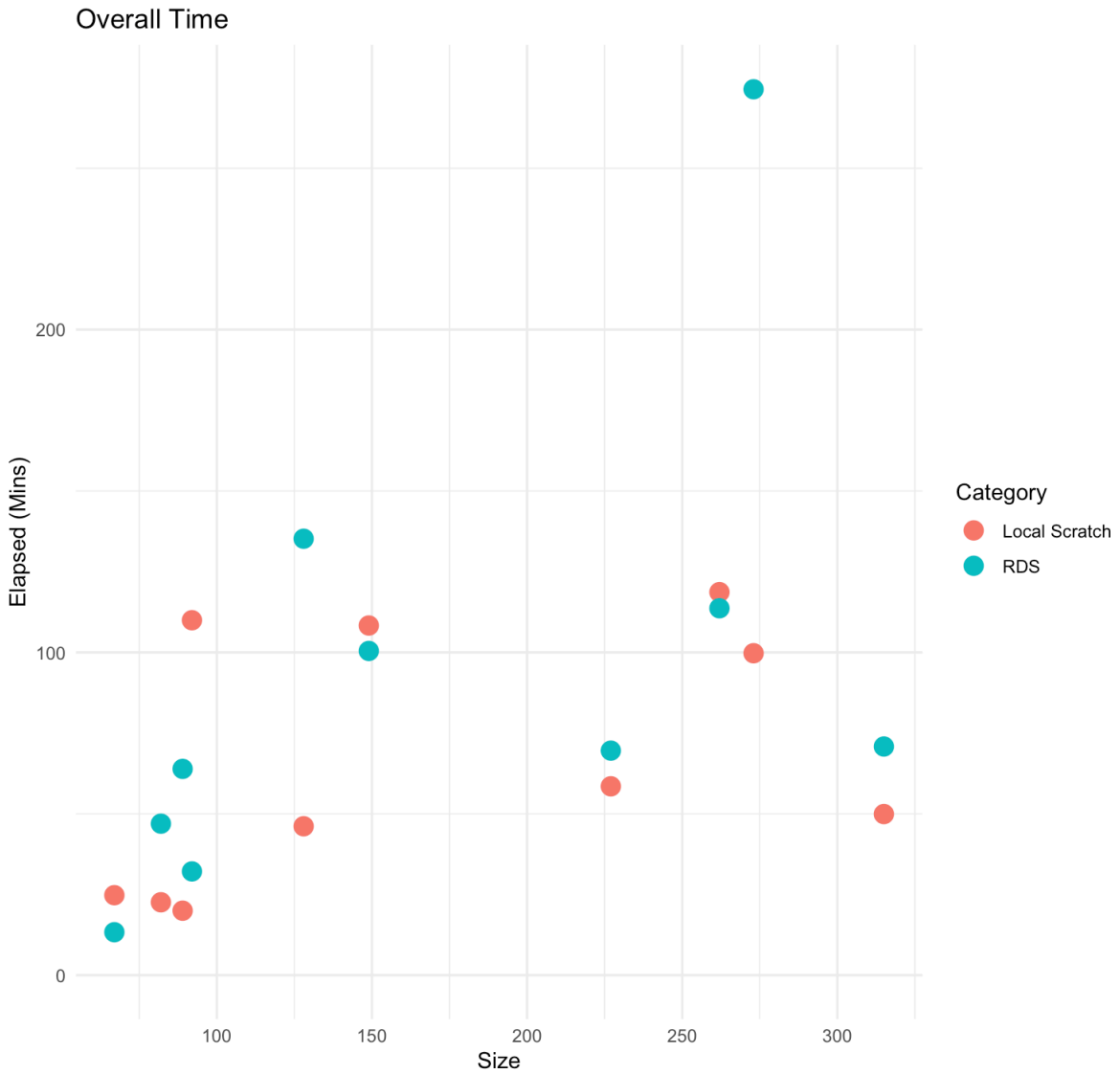
Improvements to Workflow

To test improvements, for experimentation, I've selected 10 libraries and I've taken care to include those that seem to be outliers/edge cases (using results from readcount above) while keeping variety of size in mind.

	Lib	Size
1	22KD-069F0048-1	92
2	22KD-125F0042-1	128
3	22KD-283F0020-1	89
4	22KD-350F0030-1	82
5	22KD-115F0022-1	315
6	22KD-079F0094-1	273
7	22KD-086F0081-1	262
8	22KD-122F0081-1	227
9	22KD-297F0101-1	67
10	22KD-055F0002-1	149

Implementing Local Scratch into Workflow

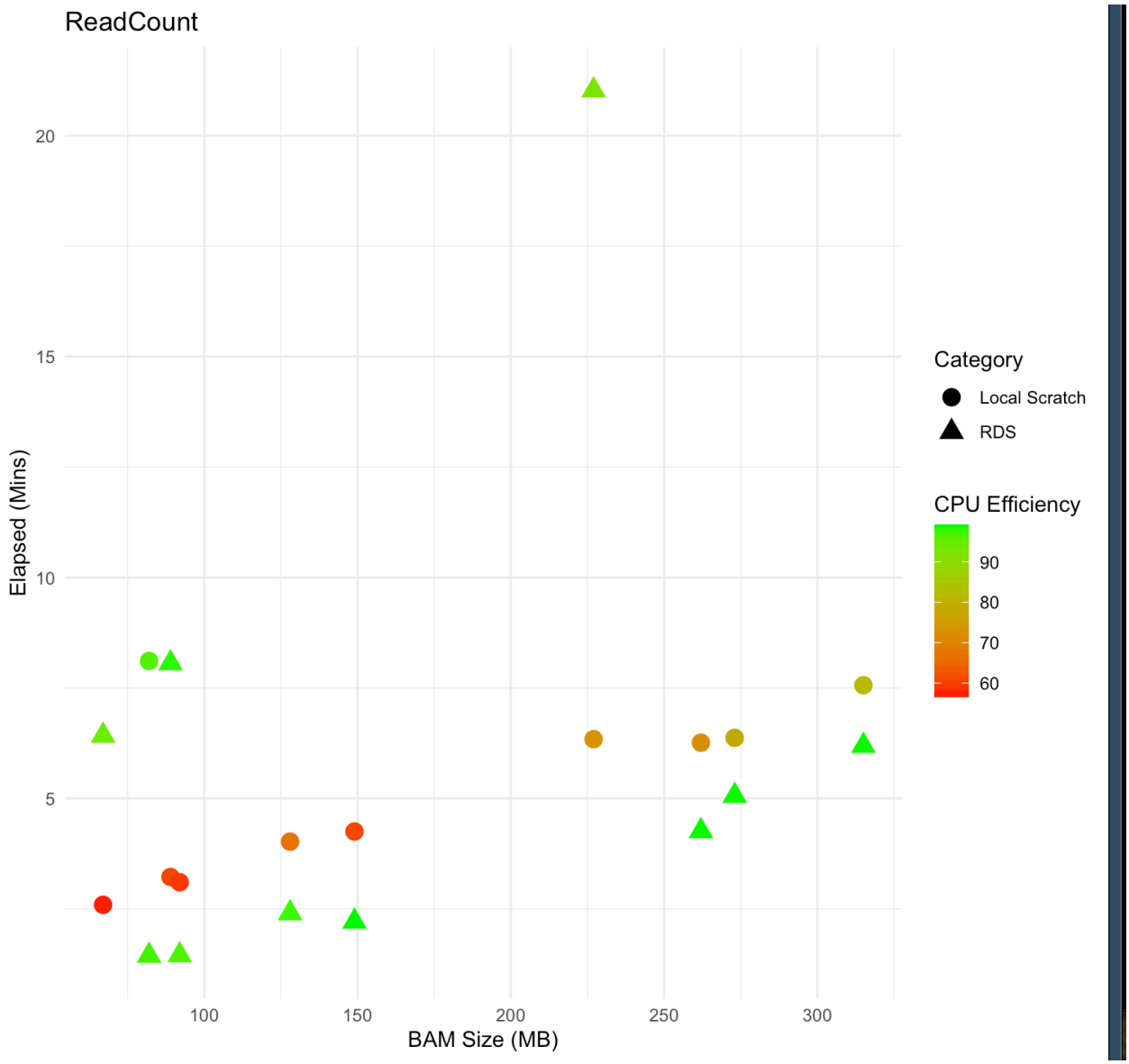
I tested on the 10 samples mentioned above, with the hope to see improvements in time and CPU efficiency. I implemented local scratch to all rules of the workflow.



Overall, there was mostly improvement. While there were a few samples where local scratch took longer, I believe it could be due to small fluctuations in how busy the cluster was at the time and how busy the assigned node was.

In particular, for the readcount rule, we are not seeing a lot of improvement in both Elapsed and CPU efficiency with local scratch, but I think that's because while local scratch saves time on I/O operations, you lose some overhead time moving files over into local scratch and moving files back into RDS. So, for a rule that doesn't take that long, you will save minimal time. If we merged it with incl_FN, for example, it could be faster as less overhead time, one less job submission and improve CPU efficiency as separately, they require minimal computation.

The most time we saved was in the annotated_readcounts rule as that is the rule that takes the most time and has many I/O operations.

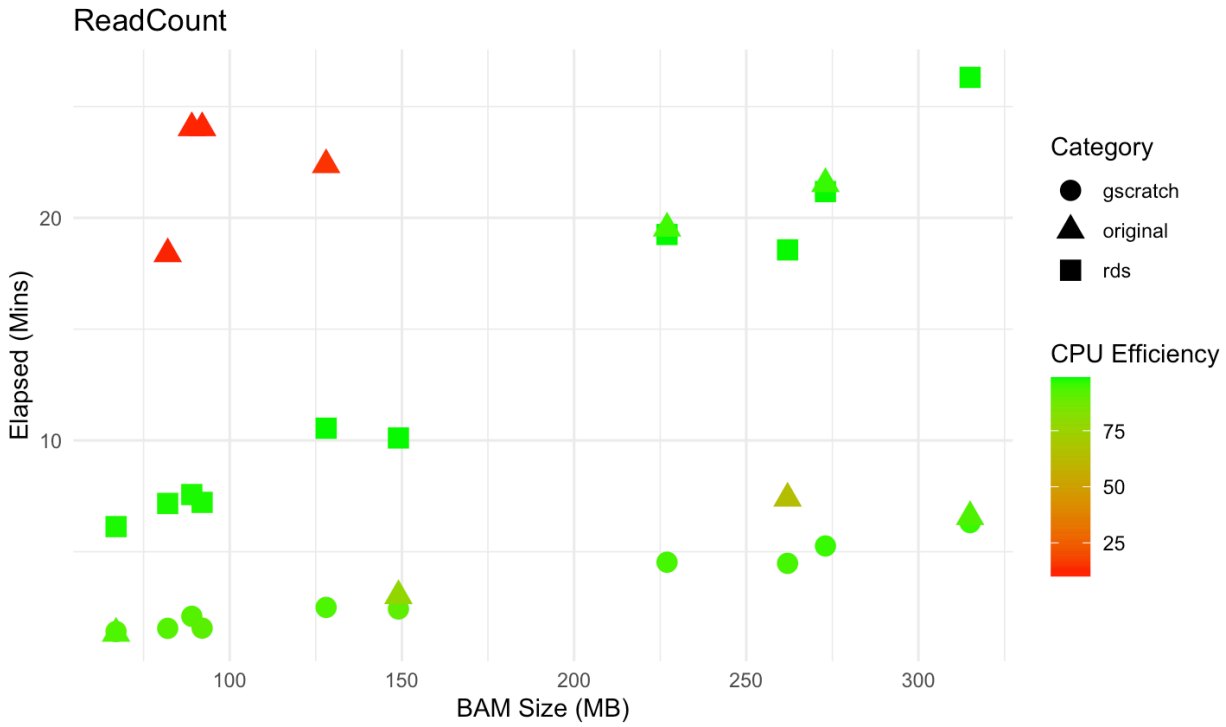


Testing Ceph with Workflow

This was to check that we can smoothly move data from the Ceph bucket when we want to analyze it. I've incorporated it into the main shell launch script where setting the argument -pull to TRUE will use my aws credentials and access the Ceph bucket. Data is copied over very quickly and will be deleted from my system once workflow is done.

Comparing gscratch with Normal Performance (while waiting for local scratch to work)

Let's compare the performance of running the 10 samples from gscratch vs running from RDS for just the readcount rule. As expected, gscratch is a lot faster which helps to improve our CPU efficiency! **We should expect an even bigger boost once we get local scratch working. (This was when local scratch wasn't working..) Moot point now.**



Implementing CleanUp

Per individual lib, there are a lot of individual files generated. We don't need all of them.
By default, per lib we want to keep these files: ***.readcount, *.all_freq, *.annotated_readcounts** (from rules readcount, calc_freq and add_annotations).
After combining, we also by default want to keep the merged result (**merged.csv**) from rule merge.
I implemented a onsuccess handler which runs a cleanup if certain conditions are met:

1. clean = True in config file
2. Snakemake workflow completes successfully. If it errors out, no cleanup happens.

You can specify which files you want to keep by adding their respective rules to the kept_files variable in config file. The kept_files variable by default has the default rules: readcount, calc_freq, add_annotations and merge.
This handler successfully worked for the 10 samples.
NOTE: This handler was selected because we want full flexibility and control over when and what cleanup should occur. For example**, even if a workflow is only partially run**, if clean = True, the handler will clean the files accordingly to the kept_files variable.

^ There is a caveat to this. Because each rule is dependent successively, Snakemake will want to regenerate the deleted intermediate files when you try to rerun to complete the workflow after a partial run.
The way around this is to specify the rule you want to run using **– force**, with the knowledge that even though we don't have the intermediate files anymore due to a clean = True, we know the files that we did keep are the latest versions.
This will also be the case if you just want to rerun any steps of the workflow with the same samples after a clean = True.
Example: I ran the workflow with clean = True and using the default kept_file variable. I want to rerun specifically the final_filt step with different filters. Snakemake will try to rerun the entire workflow to fill in the now deleted intermediate files as **it thinks all files are now not up to date. We know that is not the case** and that all we need to run is the final_filt step on the merged.csv (one of the default kept files) -> **snakemake –force final_filt**. Refer to case studies for further examples.

Whether or not clean up occurs can be seen in the **cleaned.log_file**.

Implementing Metrics File

There is a metrics file that will be produced per library and will contain the number of variants detected after main steps of the workflow. This file is continually updated as each step is ran so this can also serve to check where a library is in the workflow.
Metrics file is titled {lib}.metrics and this is its format:

# of Nucleotide Positions \t Last Updated \t readcount
# of Variants Post QC \t Last Updated \t calc_freq
# of Variants Post Filt \t Last Updated \t filt
# of Annotated Variants \t Last Updated \t annotated_readcounts
# of TET2 Variants \t Last Updated \t annotated_readcounts
# of DNMT3A Variants \t Last Updated \t annotated_readcounts

NOTE: Because this file will update as the steps occur, if a workflow step is **reran**, it will recalculate the metric for it and there will be a **new** updated timestamp.

The metrics files will be used to fill an enhanced metadata file.
We already have an existing metadata csv file (from the first 500 data) that contains information such as age, gender, library etc.
The information from the metrics file is added as new columns: NucsPos, VariantsPostQC, VariantsFiltered, AnnotatedVariants, TET2, DNM3TA in a new enhanced metadata file.
This file is titled 'enhanced_metadata.csv' for now. The script that does this is called 'add_metadata.sh'.
Run sbatch scripts/add_metadata.sh –dir=[path to results_dir] –metadata=[path to existing metadata].

NOTE: As for now, I have not incorporated 'add_metadata.sh' as part of the workflow. This is because it adds the metrics information to a new metadata file. Every time we rerun a step,

however small, it will likely update a line in the metrics file which will then prompt all the metrics to be re-added and a new enhanced metadata file to be made, so seems a bit excessive? For now, I have it set it up where you can choose to obtain an enhanced metadata file at the end of running the workflow by running 'add_metadata.sh' separately.
The metrics file per library is part of the workflow however and will not need to be generated separately.

Implementing New Method for Annovar

To allow for annotations of not just refGene (the default for annovar), I've decided to use the table_annovar.pl script instead of the annotate_variation.pl in the do_annovar step.

table_annovar.pl is an umbrella script that runs annotate_variation.pl and other scripts and outputs a nice tab-separated .txt with annotated information in the columns, with each additional column being information from an additional database. This allows for later flexibility if we want to add further databases.

In this case, I want to add in annotations from dbSNP and COSMIC (suggested by Chris).
To do this, I did these following steps:

1. Pulled data for the aforementioned databases using information from here:
   https://annovar.openbioinformatics.org/en/latest/user-guide/download/#additional-databases
   https://annovar.openbioinformatics.org/en/latest/user-guide/filter/

   dbSNP is available on annovar so I added it by doing:
   annotate_variation.pl -buildver hg19 -downdb -webfrom annovar snp138 humandb/

   For COSMIC, the latest version needs to be obtained via COSMIC itself as per the directions here: https://annovar.openbioinformatics.org/en/latest/user-guide/filter/#cosmic-annotations
   I followed the instructions they had used for cosmic100, but for the latest version: cosmic 102.

   NOTE: There is a weird duplication issue with COSMIC. I addressed it in detail in the below section. This seems to be an issue only existing with COSMIC for now, but be cautious with additional databases and check your output of do_annovar thoroughly before moving on.

2. Check that the database files are in humandb/. Database files should be named like this: {build}_{db}.txt.
3. Go to your config file and modify the value of the variable addl_db:
   "addl_db": "cosmic102,snp138".
4. Now run the do_annovar step. Check the output {lib}.{build}_multianno.txt thoroughly. Make sure that columns are separated correctly by tabs. If there is weird separation, refer to the section: **Addressing Issue with COSMIC**

Guide to Snakemake CHIP-HOP Workflow

Addressing Issue with COSMIC

I noticed a really strange issue with the data downloaded from COSMIC. There are a lot of duplicates where the Chr, Start, End, Ref and Alt are the same and the COSMIC info will be the same as well but in a different order. This messes up the resulting table and produces a table with weird tabs between each piece of COSMIC info within the COSMIC field, preventing the file from being correctly parsed.

To fix this issue:

Remove duplicates using fix_cosmic.sh. This script will go through and find duplicates by looking for the same Chr, Start, End, Ref and Alt and will then keep the one with the most occurrences listed.

Now ensure that the new file is named hg19_cosmic102.txt and in humandb/ so that it can be found by annovar.

Implementing New Script for Adding Annotations

With a new output file being produced by the new annovar method, I rewrote the script for adding annotations to our read counts. This new script is annotate_with_annovar.py. This script is now assigned to the variable annotate_bed in the config file so it can be used accordingly.

NOTE: I streamlined the method for adding the annotation information to read counts by implementing merge functions instead of a previously used for loop. This should improve computation time.

The following output file *.annotated_readcounts has this column format:

library	chr	pos	ref	alt	depth	count	vaf	avg_BQ	avg_pos_as_fraction
	type	class	gene	func	cosmic102	snp138			

^ cosmic102 and snp138 are the additional databases we wanted to pull information from. Additional databases will be represented as a new column always.

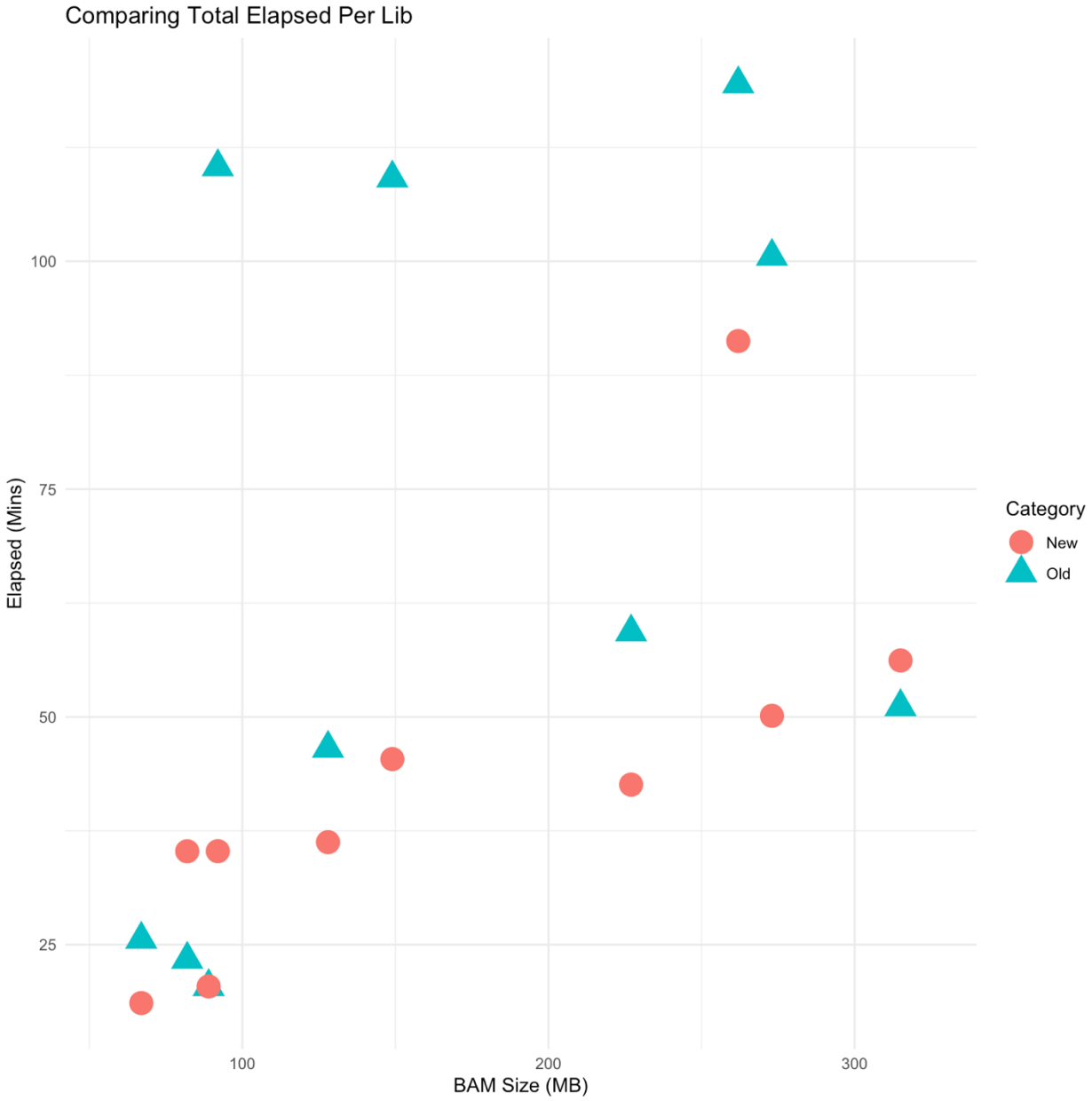
NOTE: From comparing this new annotated output file to the previous one, one immediate change I've noticed is there are a few more mutations being annotated now. This is the case because there was a bug in the previous one.

The bug would mess up the spacing of the columns for rows when annovar found the type of mutation to be stopgain, stopless, startles, unknown (as according to refGene) as seen in the *.EVF. These rows had one less column because they had no 'class' column and therefore were incorrectly parsed during add_annotations, resulting in them being missed.

I've adjusted for this by adding in a placeholder '.' to fill in the class column thereby allowing it to be parsed correctly.

To see the impact on computation time, I compared the updated workflow as of 08/07/2025 and the original workflow on the aforementioned 10 libraries. We see large improvement in elapsed for most of the libraries. There were a few libraries that had worse computation time with the

updated workflow. Not entirely sure of the reason but I suspect it has to do with the combination of how busy the cluster is, and the addition of running metrics and cleaning files up that could have tacked on additional time for those libraries. I'm not concerned because the cases where New took longer, the difference was not as substantial as the improvement seen in the other libraries.



Running the VC Workflows

Pulling Data from Ceph

We will be storing the raw data (bams) in a Ceph bucket. We will copy the data over when we want to use it.
To access the data, we will specify the data directory we want to pull the data to: -d [dir] when running the launch script: run_pipeline.sh
We also need you to specify the aws profile (-p) you need to access Ceph.

NOTE: The -d should be the same as the bams variable in the config file.
If you don't specify -d, the script will assume that you don't want to pull any data and you already have the data you need.

Config File Breakdown

There are 2 config files. config_variantcalling.json corresponds to 02_variantcalling_manual.smk. config_vc_vep.json corresponds to 02_variantcalling_BCF_VEP.smk.
Make sure to use the appropriate config file for the workflow you're running.

Always check the config file's variables and change as desired.

config_variantcalling.json

master_dir	Main directory where analysis will take place
ref	Reference genome
bams	Directory containing indexed sorted bam files
results	Directory where results from workflow will be stored
regions	Reference target regions to assist with bam read-count tool
build	Reference genome build
clean	Boolean variable to determine if you want intermediate files to be deleted
kept_rules	Set which rules whose output files you want to keep
min_depth	Minimum depth/coverage
max_depth	Maximum depth/coverage
min_vaf	Minimum variant allele frequency
max_vaf	Maximum variant allele frequency
min_supp_reads	Minimum number of supporting reads
minBQ	Minimum base quality score
readcount	Script used to run bam read-count tool in **readcount** step
calc_freq	Script used to run readcount2freq

readcount2freq	Script to calculate VAF for each variant during **calc_freq** step
filter	Script to perform initial filtering during **filt** step
format_annovar	Script to run freq2bed
freq2bed	Script to change the format of the variants to be annovar friendly during **format_annov** step
do_annovar	Script to run annovar for provided variants during the **do_annovar** step
addl_db	Additional filter-based databases you want annotation information from
annovar	Directory that contains annovar tool
add_annotations	Script to run annotate_bed
annotate_bed	Script to add annotations from annovar to corresponding variants for **add_annotations** step

*Highlighted in Yellow = Steps likely to be Changed

config_vc_vep.json

master_dir	Main directory where analysis will take place
ref	Reference genome
bams	Directory containing indexed sorted bam files
results	Directory where results from workflow will be stored
regions	Reference target regions for variant calling
min_map_q	Minimum mapping quality of reads
min_base_q	Minimum base quality
vep_cache	Directory where VEP cache files are stored
min_depth	Minimum depth/coverage
min_var_q	Minimum variant quality
min_gq	Minimum genotype quality
max_af	Maximum population allele frequency from gnomad
gnomad_ref	Reference gnomad annotation (must match ref genome version)
build	Reference genome build
vep_t	Number of threads to use in vep
bcf_call	Script used to run variant calling in **bcf_call** step
bcf_filter	Script to filter variant calling by quality filters in **bcf_filt** step
prefilt_w_gnomad	Script to perform annotation using gnomad and prefiltering by AF in **prefilt_w_gnomad** step
annotate_vep	Script to annotate using vep in **annotate_vep** step

*Highlighted in Yellow = Steps likely to be Changed
*Highlighted in Blue = Refer to below section **Confirm Chromosome Format** to confirm these variables! Be careful!

Guide to Snakemake CHIP-HOP Workflow

Confirm Chromosome Format

It is important to check that your input bams, genome and gnomad reference have the same chromosome format.
To check your format for bams do: samtools view -H <absolute path to bam>.
Usually, your format will be in chr# or just #.
You must ensure that the reference genome and gnomad you use have the same chromosome format.
If you're using the human genome hg19, you are free to use my files. They are stored in: /home/groups/CEDAR/chaoe/projects/HOP/ref_files (subject to change to boniface)

If you need to adjust your vcf file to chr# or #, you can utilize bcf annotate –rename-chrs. For example:
bcftools annotate \ --rename-chrs chr_map.txt \ -Oz -o "$out" \ "$in" bcftools index "$out"

Also, make sure that your genome build is correct and in line with the input bams, genome and gnomad reference files you are using.

Cluster Generic Profile Breakdown

You will also want to check the cluster generic profile to confirm the desired cluster config settings.
There are 2 cluster config profile files.
config/cluster/config.v8+.yaml corresponds to 01_preprocessing.smk and 02_variantcalling_manual.smk.
config/cluster_updated/config.v8+.yaml corresponds to 02_variantcalling_BCF_VEP.smk.

Make sure to use the appropriate config file for the workflow you're running.

config/cluster/config.v8+.yaml

partition	batch
account	cedar
cpus-per-task	{threads} aka set to depend on what's specified in Snakefile, otherwise default cpu=1
mem	{resources.mem}
job-name	smk-{rule}-{wildcards}
output	<log folder>/{rule}/{rule}-{wildcards}-%j.out
gres	{resources.gres} aka set to depend on what's specified in Snakefile, otherwise default gres="disk:1024" bc we want local scratch
time	{resources.time} aka set to depend on what's specified in Snakefile, otherwise default time="01:00:00"
cluster-generic-status-cmd	python ./scripts/cluster-status.py
jobs	40 (40 submitted at same time)

rerun-incomplete	True
use-conda	True

*Highlighted in Yellow = Steps likely to be Changed
** the cluster status script is used to let Snakemake know if a SLURM job timed out/cancelled/failed etc.

config/cluster_updated/config.v8+.yaml

partition	batch
account	cedar
cpus-per-task	{threads} aka set to depend on what's specified in Snakefile, otherwise default cpu=1
mem	{resources.mem}
job-name	smk-{rule}-{wildcards}
output	<log folder>/{rule}/{rule}-{wildcards}-%j.out
gres	{resources.gres} aka set to depend on what's specified in Snakefile, otherwise default gres="disk:1024" bc we want local scratch
time	{resources.time} aka set to depend on what's specified in Snakefile, otherwise default time="01:00:00"
cluster-generic-status-cmd	python ./scripts/cluster-status.py
jobs	40 (40 submitted at same time)
rerun-incomplete	True
use-conda	True
latency-wait	60

*Highlighted in Yellow = Steps likely to be Changed
** the cluster status script is used to let Snakemake know if a SLURM job timed out/cancelled/failed etc.

NOTE: I added latency-wait here because I noticed that snakemake was incorrectly marking jobs as unfinished because the outputs were taking a while to populate. I have it set to wait for a minute here.

Conda Environments

HOP

For 01_preprocessing.smk and 02_variantcalling_manual.smk, use this environment called HOP. This environment is simple and only contains the packages and dependencies required to run these workflows and nothing more.
This environment is contained within HOP.yaml.

You do need to install the environment if you want to use it. Install by doing: conda env create -f HOP.yaml.

vep_HOP

For 02_variantcalling_BCF_VEP.smk, use the vep_HOP environment. This environment is simple and only contains the packages and dependencies required to run this workflow and nothing more.
Install this environment by doing: conda env create -f vep_HOP.yaml.

Launching the Workflow

The run_pipeline.sh script is used to launch the snakemake workflow.
NOTE: Always confirm your snakemake command in run_pipeline.sh by performing a dry run first (snakemake -n).
Here are some example commands for dry run:

Run entire workflow:

```
snakemake -n -s $snakefile --profile=$clusterfolder --configfile=$config  --forcerun all
```

Run until rule (*):

```
snakemake -n -s $snakefile  --profile=$clusterfolder --configfile $config --until (*)
```

Run only a specific rule (*):

```
snakemake -n -s $snakefile --profile $clusterfolder --configfile $config --force (*) –until (*)
```

Now to launch the workflow, you'll run sbatch run_pipeline.sh with the following parameters:

-c	Path to config file	Required
-env	Name of conda environment	Required
-d	Path of directory to receive data from Ceph bucket	Only required if pulling data from Ceph
-s3_p	S3 Profile name	Only required if pulling data from Ceph
-clust_p	Path to cluster config profile	Required
-u	Name of rule to run workflow until	Only required if wanting to run until specific rule
--cores	Number of cores for snakemake workflow (--cores)	Only required if wanting to allocate total number of cores in workflow (Default: 6)
-s	Path to snakemake workflow	Required
-f	Name of rule to forcefully run	Only required if wanting to forcefully run specific rule

Guide to Snakemake CHIP-HOP Workflow

Case Studies

1. Run the manual mutation calling workflow entirely through once, setting clean = True and keeping the default files. I already have my data in a directory.

```
snakefile=<path to 02_variantcalling_manual.smk>
config=<path to config_variantcalling.json >
clusterfolder=<path to config/cluster>

snakemake -n -s $snakefile --profile=$clusterfolder --configfile=$config

sbatch run_pipeline.sh -s $snakefile -c $config -clust_p $clusterfolder -env HOP
```

   Now we want to run again, starting from the filt step, with some filters changed.

```
snakemake -n -s $snakefile --profile=$clusterfolder --configfile=$config
--forcerun filt

sbatch run_pipeline.sh -s $snakefile -c $config -clust_p $clusterfolder -env HOP
-f filt
```

   Now rerun the final_filter step, with different filters.

```
snakemake -n -s $snakefile --profile=$clusterfolder --configfile=$config
--forcerun final_filt

sbatch run_pipeline.sh -s $snakefile -c $config -clust_p $clusterfolder -env HOP
-f final_filt
```

2. Run the manual mutation calling workflow partially up to filt setting clean = False for now. I already have my data in a directory.

```
snakefile=<path to 02_variantcalling_manual.smk>
config=<path to config_variantcalling.json >
clusterfolder=<path to config/cluster>

snakemake -n -s $snakefile --profile=$clusterfolder --configfile=$config
--until filt

sbatch run_pipeline.sh -s $snakefile -c $config -clust_p $clusterfolder -env HOP
-u filt
```

   Now run the rest, setting clean = True and keeping default files.

```
snakemake -n -s $snakefile --profile=$clusterfolder --configfile=$config --
forcerun all

sbatch run_pipeline.sh -s $snakefile -c $config -clust_p $clusterfolder -env HOP
-f all
```

3. Pull data from Ceph bucket to data directory and run the manual mutation calling workflow entirely through.

```
snakefile=<path to 02_variantcalling_manual.smk>
config=<path to config_variantcalling.json >
clusterfolder=<path to config/cluster>


sbatch run_pipeline.sh -s $snakefile -clust_p clusterfolder –c $config -d $data_dir
-s3_p $profile -env HOP
```

4. Run the updated mutation calling workflow (bcftools and vep) in its entirety.

```
snakefile=<path to 02_variantcalling_BCF_VEP.smk>
config=<path to config_vc_vep.json>
clusterfolder=<path to config/cluster_updated>

snakemake -n -s $snakefile --profile=$clusterfolder --configfile=$config

sbatch run_pipeline.sh -s $snakefile -clust_p clusterfolder –c $config
-env vep_HOP
```

Other Information

I've created a separate script for analyzing the annotated dataframe, comparing the mutation calls to lists of known calls and producing companion visualizations.
This script is called AnalyzeMutCalls.py.
The script to create custom calls is called ConvertCalls.py.
The config file that controls the parameters for AnalyzeMutCalls.py is called config_annot.json.

Here's the breakdown of the config file:

infile	Absolute path to annotated dataframe (produced from workflow)
calls	Named list of absolute paths to known calls in txt files (if absent, refer to ConvertCalls.py)
annot	Absolute path to annotated dataframe (outsourced)
metadata	Absolute path to patient metadata information
dbsnp	Absolute path to known sites in dbsnp

Guide to Snakemake CHIP-HOP Workflow

plt	Absolute path to directory where plots will go
ver	Can specify version of plots if we are running multiple runs
minvaf	Minimum VAF
maxvaf	Maximum VAF
mincount	Minimum read count
dict	Absolute path to dictionary of calls (if absent, can be custom made)

*Highlighted in Yellow = Steps likely to be Changed
*Highlighted in Blue= One or the other is necessary